- Event Handling
- Java IO

- An **event** can be defined as changing the state of an object or behavior by performing actions.

- The **java.awt.event** package can be used to provide various event classes.

- **Classification of Events**
  - Foreground Events
  - Background Events

- **Event Delegation Model**
  - Sources: Events are generated from the source.
  - Listeners: Listeners are used for handling the events generated from the source.

Listener Interface	Methods
ActionListener	• actionPerformed()
AdjustmentListener	• adjustmentValueChanged()
ComponentListener	• componentResized() • componentShown() • componentMoved() • componentHidden()
ContainerListener	• componentAdded() • componentRemoved()
FocusListener	• focusGained() • focusLost()
ItemListener	• itemStateChanged()
KeyListener	• keyTyped() • keyPressed() • keyReleased()

Listener Interface	Methods
MouseListener	• mousePressed() • mouseClicked() • mouseEntered() • mouseExited() • mouseReleased()
MouseMotionListener	• mouseMoved() • mouseDragged()
MouseWheelListener	• mouseWheelMoved()
TextListener	• textChanged()
WindowListener	• windowActivated() • windowDeactivated() • windowOpened() • windowClosed() • windowClosing() • windowIconified() • windowDeiconified()

```
import java.awt.*;
import java.awt.event.*;
class Event extends Frame implements ActionListener{
TextField tf;
Event(){
tf=new TextField();
tf.setBounds(60,50,170,20);
Button b=new Button("click me");
b.setBounds(100,120,80,30);
b.addActionListener(this);
add(b);add(tf);
setSize(400,400);
setLayout(null);
setVisible(true);
}
public void actionPerformed(ActionEvent e){
tf.setText("Welcome to Java Course");
}
public static void main(String args[]){
Event e=new Event();
}
}
```

Example

```
import javax.swing.*;
import java.awt.*;
public class SetTest {
   public static void main(String arg[]) {
      JFrame frame = new JFrame("SetBounds Method Test");
      frame.setSize(375, 250);

      frame.setLayout(null);

      JButton button = new JButton("Hello Java");

      button.setBounds(80,30,120,40);
      frame.add(button);
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

      frame.setVisible(true);
   }
}
```

```
import java.awt.*;
class Check
{
      Check() {
      Frame f = new Frame("Checkbox Example");
       Checkbox checkbox1 = new Checkbox("C++“, true);
       checkbox1.setBounds(100, 100,  50, 50);
       Checkbox checkbox2 = new Checkbox("Java");

checkbox2.setBounds(100, 150,  50, 50);
       f.add(checkbox1);
       f.add(checkbox2);
       f.setSize(400,400);
       f.setLayout(null);
       f.setVisible(true);
    }
public static void main (String args[])
{
   new Check();
}
}
```

- **Java I/O** (Input and Output) is used *to process the input* and *produce the output*.
- Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations.
- We can perform **file handling in Java** by Java I/O API.

Streams

- A stream is a sequence of data.
- Most commonly used streams are :

  1) System.out: standard output stream

  2) System.in: standard input stream

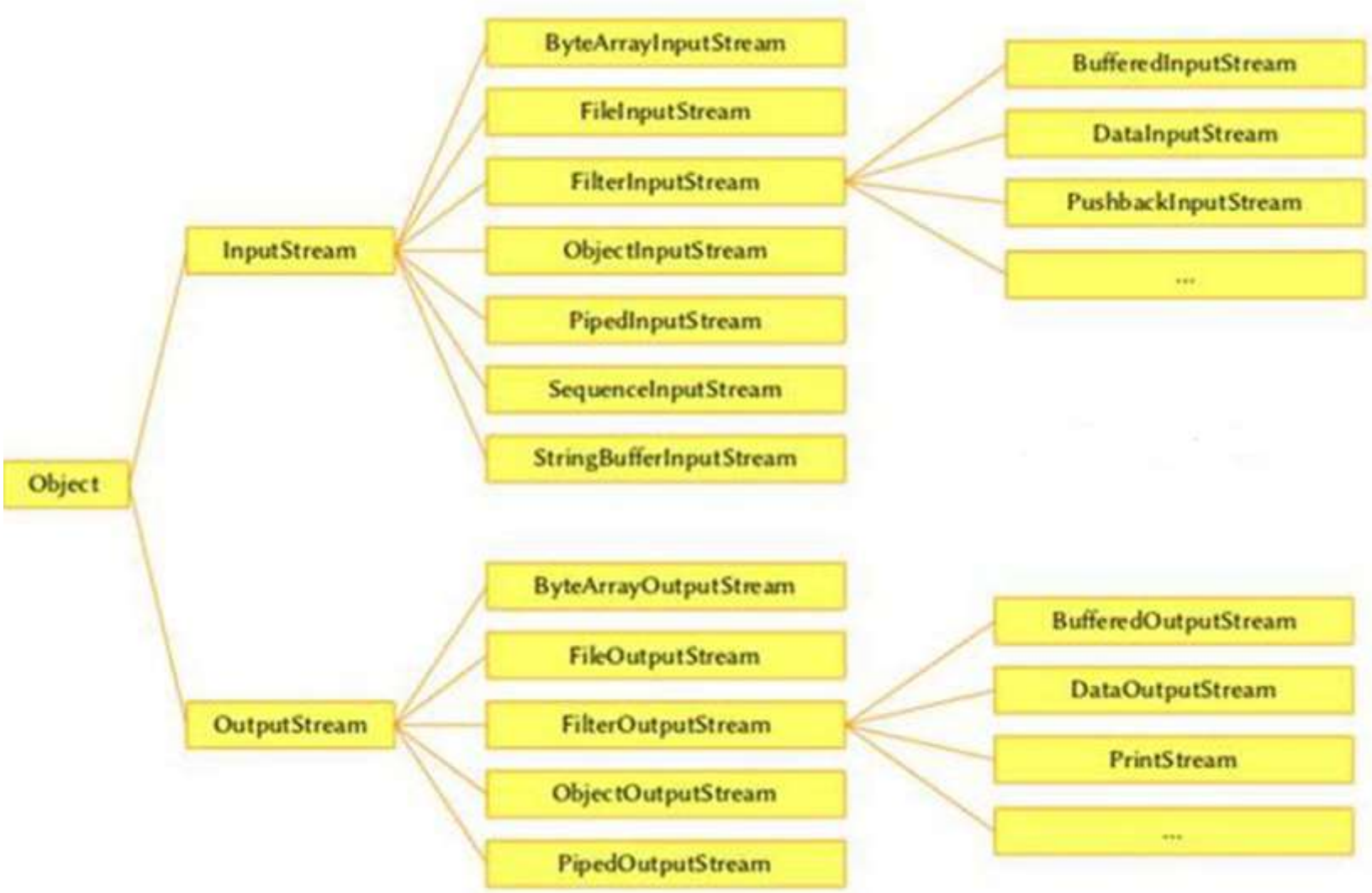
  3) System.err: standard error stream

Example

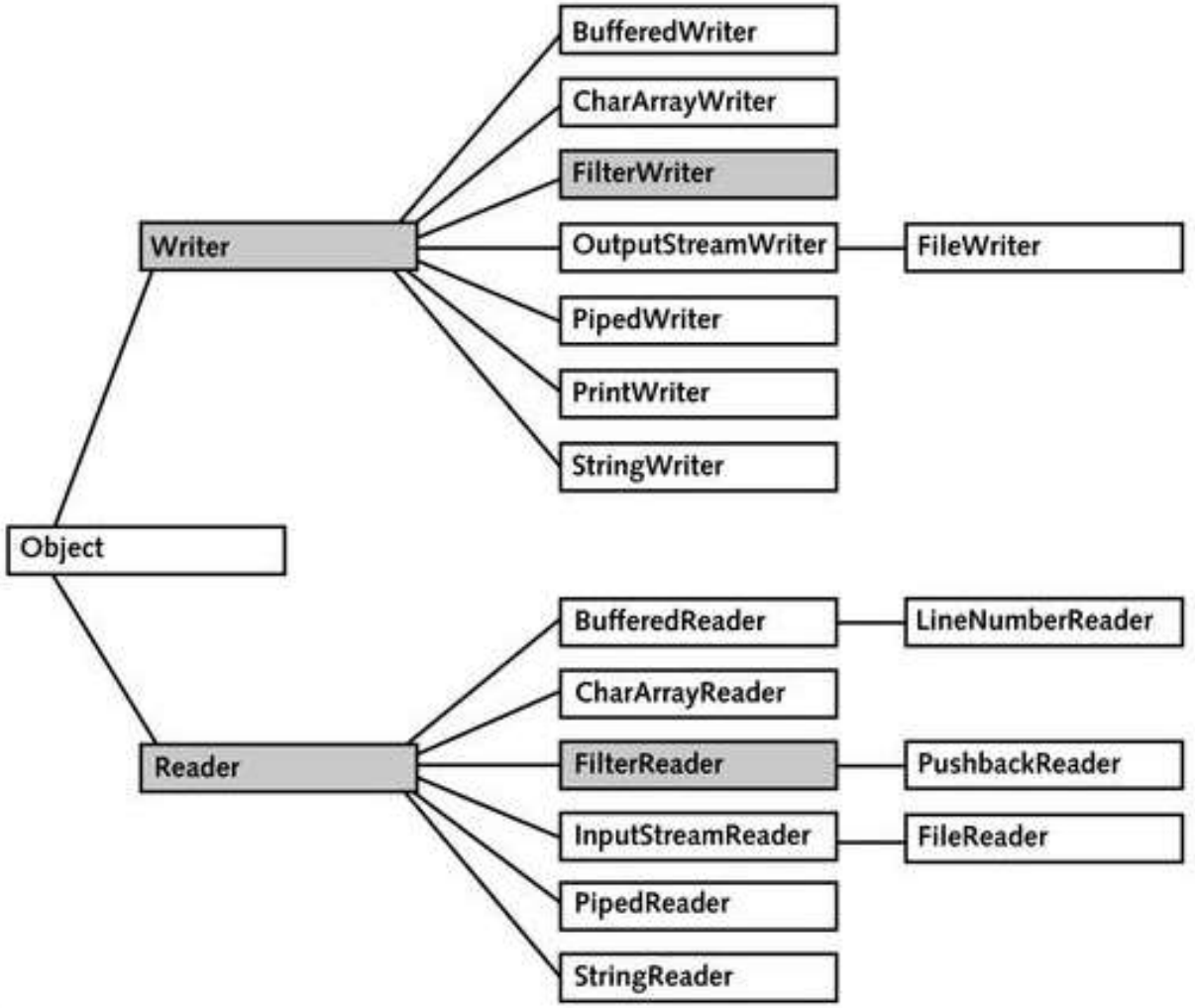
```
import java.io.*;
class Test{
public static void main(String a[]) throws IOException
{
int i=System.in.read();
System.out.println("Welcome");
System.err.println("Error");
System.out.println(i);
}
}
```

A
Welcome
Error
65

Hierarchy of Java IO Stream



Hierarchy of Java Reader/Writer



OutputStream class

- An output stream accepts output bytes and sends them to some sink.

Method	Description
1) public void write(int) throws IOException	is used to write a byte to the current output stream.
2) public void write(byte[]) throws IOException	is used to write an array of byte to the current output stream.
3) public void flush() throws IOException	flushes the current output stream.
4) public void close() throws IOException	is used to close the current output stream.

InputStream class

Method	Description
1) public abstract int read() throws IOException	reads the next byte of data from the input stream. It returns -1 at the end of the file.
2) public int available() throws IOException	returns an estimate of the number of bytes that can be read from the current input stream.
3) public void close() throws IOException	is used to close the current input stream.

Example of InputStream

```
import java.io.*;
class Test1{
public static void main(String a[]) throws IOException{
InputStream fin=new FileInputStream(“abc.txt");
int i=fin.read();
System.out.println((char) i);
fin.close();
}
}
```

```
import java.io.*;
class Test2{
public static void main(String a[]) throws IOException{
InputStream fin=new FileInputStream(“abc.txt");
int i;
while( (i= fin.read())!=-1)
{
System.out.print((char) i);
}
fin.close();
}
}
```

Example of FileInputStream

```
import java.io.*;
class Test5{
public static void main(String a[]) throws IOException{
FileInputStream f=new FileInputStream(“abc.txt");
System.out.println(f.available());
System.out.println("char: "+(char) f.read());
System.out.println("char: "+(char) f.read());
System.out.println("char: "+(char) f.read());
f.skip(1);
System.out.println("char: "+(char) f.read());
System.out.println("char: "+(char) f.read());
f.skip(3);
System.out.println("char: "+(char) f.read());
f.close();
}
}
```

Example of OutputStream

```
import java.io.*;
class Test4{
public static void main(String a[]) throws IOException {
OutputStream f=new FileOutputStream(“abc.txt");
f.write('b’);
f.flush();
f.close();
}
}
```

```
import java.io.*;
class Test3{
public static void main(String a[]) throws IOException{
OutputStream f=new FileOutputStream("D:\\myfil.txt");
String s="Welcome New Year";
byte b[]=s.getBytes();
f.write(b);
f.flush();
f.close();
}
}
```

Example of FileOutputStream

```
import java.io.*;
class Test7{
public static void main(String a[]) throws IOException{
FileOutputStream f=new FileOutputStream("D:\\myfil.txt");
byte b[]={65,66,67,68,69};
f.write(b);
f.flush();
System.out.println(f.getChannel());
System.out.println(f.getFD());
f.close();
}
}
```

Example of BufferedOutputStream

```
import java.io.*;
public class BufferedOutput{
public static void main(String args[])throws Exception{
   FileOutputStream f=new FileOutputStream("test1.txt");
    BufferedOutputStream b=new BufferedOutputStream(f);
    String s="Welcome to java Programming course";
    byte b1[]=s.getBytes();
    b.write(b1);
    b.flush();
    b.close();
    f.close();
    }
}
```

Example of SequenceInputStream

```
import java.io.*;
class Input1{
public static void main(String args[])throws Exception{
   FileInputStream fin1=new FileInputStream(“Test.txt");
   FileInputStream fin2=new FileInputStream("abc.txt");
   FileOutputStream fout=new FileOutputStream("tes.txt");
   SequenceInputStream sis=new   SequenceInputStream(fin1,fin2);
   int i;
   while((i=sis.read())!=-1)
   {
     fout.write(i);
   }
   sis.close();
   fout.close();
   fin1.close();
   fin2.close();
   System.out.println("Success..");
  }
}
```

File I/O

- The File class is Java's representation of a file or directory path name. Because file and directory names have different formats on different platforms, a simple string is not adequate to name them.

- File class Constructor:
  - File f = new File(String pathname) ;
    - Creates a new File instance by converting the given pathname string into an abstract pathname.
  - File f = new File(File parent, String child);
    - Creates a new File instance from a parent abstract pathname and a child pathname string.
  - File f = new File(String parent, String child);
    - Creates a new File instance from a parent pathname string and a child pathname string.

```
import java.io.*;
class CharRead1{
public static void main(String a[]) throws IOException{
File f=new File ("D://File.txt”);
System.out.println(f.exists());
f.createNewFile();
System.out.println(f.exists());
}
}
```

```
import java.io.*;
class CharRead2{
public static void main(String a[]) throws IOException{
String fname=a[0];
File f=new File (fname);
System.out.println(f.exists());
f.createNewFile();
System.out.println(f.exists());
}
}
```

```
import java.io.*;
class Test{
public static void main(String a[]) throws IOException{
File f=new File ("JS");
System.out.println(f.exists());
f.mkdir();
File f1=new File("JS","abc.txt");
//File f1=new File(f,"abc.txt");
f1.createNewFile();
System.out.println(f1.exists());
}
}
```

Important methods of File class with Example

```
import java.io.*;
class Content
{
   public static void main(String[] args) throws IOException {
      String fname =args[0];
      File f = new File(fname);
      System.out.println("File name :"+f.getName());
      System.out.println("Path: "+f.getPath());
      System.out.println("Absolute path: " +f.getAbsolutePath());
      System.out.println("Parent: "+f.getParent());
      System.out.println("Exists :"+f.exists());
      if(f.exists())
      {
         System.out.println("Is writeable :"+f.canWrite());
         System.out.println("Is readable :"+f.canRead());
         System.out.println("Is a directory:"+f.isDirectory());
         System.out.println("Is a File :"+f.isFile());
         System.out.println("File Size in bytes : "+f.length());
      }
   }
```

FileWriter class

- Java FileWriter class is used to write character-oriented data to a file. It is character-oriented class which is used for file handling in java.

- Unlike FileOutputStream class, we don't need to convert string into byte array because it provides method to write string directly.

- Constructors used in FileWriter class:

- FileWriter f=new FileWriter(String name);
- FileWriter f=new FileWriter(File f1);
- FileWriter f=new FileWriter(String name, boolean append);
- FileWriter f=new FileWriter(File f1, boolean append);

Methods of FileWriter class

Method	Description
void write(String text)	It is used to write the string into FileWriter.
void write(int c)	It is used to write the char into FileWriter.
void write(char[] c)	It is used to write char array into FileWriter.
void flush()	It is used to flushes the data of FileWriter.
void close()	It is used to close the FileWriter.

Example of FileWriter

```
import java.io.FileWriter;

public class Main {

  public static void main(String args[]) throws Exception {

      FileWriter output = new FileWriter("output.txt“, true);
      String data = "This is the data in the output file";
      output.write(data);
     output.write('\n');
     output.write(100);
     output.write('\n');
     char c[]={'a','c','d'};
     output.write(c);
     output.flush();
     output.close();
   }

   }
```

BufferedWriter class

- Java BufferedWriter class is used to provide buffering for Writer instances. It makes the performance fast.
- It inherits the Writer class.

- Constructors used in BufferedWriter class:

- BufferedWriter f=new BufferedWriter(Writer name);

- BufferedWriter f=new BufferedWriter(Writer name, int size);

Methods of BufferedWriter class

Method	Description
void newLine()	It is used to add a new line by writing a line separator.
void write(int c)	It is used to write a single character.
void write(char[] cbuf, int off, int len)	It is used to write a portion of an array of characters.
void write(String s, int off, int len)	It is used to write a portion of a string.
void flush()	It is used to flushes the input stream.
void close()	It is used to closes the input stream

Example of BufferedWriter

```
import java.io.*;

public class Main3 {

  public static void main(String args[]) throws Exception {

      FileWriter fw = new FileWriter("output1.txt");
      BufferedWriter b=new BufferedWriter(fw);
      String data = "This is the data in the output file";
      b.write(data,3,9);
     b.newLine();
     b.write(100);
     b.newLine();
     char c[]={'a','c','d'};
     b.write(c,0,2);
     b.flush();
     b.close();
   }

   }
```

PrintWriter class

- Unlike other writers, PrintWriter converts the primitive data (int, float, char, etc.) into the text format. It then writes that formatted data to the writer.

- Constructors used in PrintWriter class:

- PrintWriter P=new PrintWriter(String name);

- PrintWriter P=new PrintWriter(File f);

- PrintWriter P=new PrintWriter(Writer name);

```
import java.io.*;

public class Main4 {

  public static void main(String args[]) throws Exception {

      PrintWriter b=new PrintWriter("file.txt");
      String data = "This is the data in the output file";
      b.print("data");

     b.print(100);
     b.println(10.5);

     b.println(true);
                b.write(100);
     b.flush();
     b.close();
    }

    }
```

FileReader class

- Java FileReader class is used to read data from the file.
- Constructors used in FileReader class:
- FileReader f=new FileReader(String name);
- FileReader f=new FileReader(File f);

Methods of FileReader class

- read() - reads a single character from the reader
- read(char[] array) - reads the characters from the reader and stores in the specified array.
- read(char[] array, int start, int length) - reads the number of characters equal to length from the reader and stores in the specified array starting from the position start.
- close()

Example of FileReader

```
import java.io.FileReader;
public class FileReaderExample {
    public static void main(String args[])throws Exception{
        FileReader fr=new FileReader("D:\\tes.txt");
        int i;
        while((i=fr.read())!=-1)
        System.out.print((char)i);
        fr.close();
    }
}
```

BufferdReader class

- Java BufferedReader class is used to read the text from a character-based input stream.
- It can be used to read data line by line by readLine() method. It makes the performance fast.
- It inherits Reader class

- Constructor used in BufferdReader Class
  - BufferedReader f=BufferedReader(Reader rd);
  - BufferdReader f= BufferedReader(Reader rd, int size);

Methods of BufferedReader class

Method	Description
int read()	It is used for reading a single character.
int read(char[] cbuf, int off, int len)	It is used for reading characters into a portion of an aray.
boolean markSupported()	It is used to test the input stream support for the mark and reset method.
String readLine()	It is used for reading a line of text.
boolean ready()	It is used to test whether the input stream is ready to be read.
long skip(long n)	It is used for skipping the characters.
void reset()	It repositions the stream at a position the mark method was last called on this input stream.
void mark(int readAheadLimit)	It is used for marking the present position in a stream.
void close()	It closes the input stream and releases any of the system resources associated with the stream.

Example of BufferedReader

```
import java.io.*;
public class Buffered{
public static void main(String args[])throws Exception{
    FileReader r=new FileReader("output.txt");
    BufferedReader b=new BufferedReader(r);
    int i;
    while((i=b.read())!=-1)
   System.out.print((char) i);
   b.close();
   r.close();
   }
   }
```

Example of InputStreamReader

```
import java.io.*;
class CharRead{
public static void main(String a[]) throws IOException{

BufferedReader b=new BufferedReader (new InputStreamReader(System.in));

char ch=(char) b.read();
System.out.println(ch);
}
}
```

```
import java.io.*;
public class Buffered{
public static void main(String args[])throws Exception{
    BufferedReader b=new BufferedReader(new InputStreamReader (System.in));
    String name=" ";
   System.out.println("Enter data: ");
    while(!name.equals("stop")){
     System.out.println("Enter data: ");
     name=b.readLine();
     System.out.println("data is: "+name);
    }
   b.close();
```

Byte Stream v/s Character Sytream

- Characters are stored using Unicode conventions. Character stream automatically allows us to read/write data character by character.

- Byte streams process data byte by byte (8 bits).

- Character stream is useful when we want to process text files. These text files can be processed character by character. A character size is typically 16 bits.

- Byte oriented reads byte by byte. A byte stream is suitable for processing raw data like binary files.

- Names of character streams typically end with Reader/Writer and names of byte streams end with InputStream/OutputStream.

THANK YOU…